Time Series and Recurrent Networks

ES 335 Machine Learning, Semester I 2025–26 Sources: lectures "Forecasting" and "Recurrent neural networks", cheat sheet ML28, and the autoregressive model notebook

What this file is really about:

$$\text{look at the series} \rightarrow \text{beat the baselines} \rightarrow \text{split in time} \rightarrow \text{regress on lags} \rightarrow \text{learn a memory}$$

How to use this file. Part A builds forecasting from simple rules to recurrent networks. Part B shows the code. Part C is practice with answers.

Step-up path

You need. Linear regression and the idea of a held-out set.

1. Read trend, season and noise.
2. Beat simple baselines.
3. Validate forward in time.
4. Regress on lags.
5. Let a recurrent network learn a memory, and see why gradients vanish.

You can stop when you can forecast honestly and explain why LSTMs were invented.

Contents

Part A Maths From the Ground Up

A1 What makes a series forecastable

Forecasting depends on how well we understand the drivers, how much data we have, and whether the forecast itself changes the outcome. Electricity demand is forecastable. A stock price is hard, partly because the market reacts to any good forecast.

A2 Trend, season and what is left

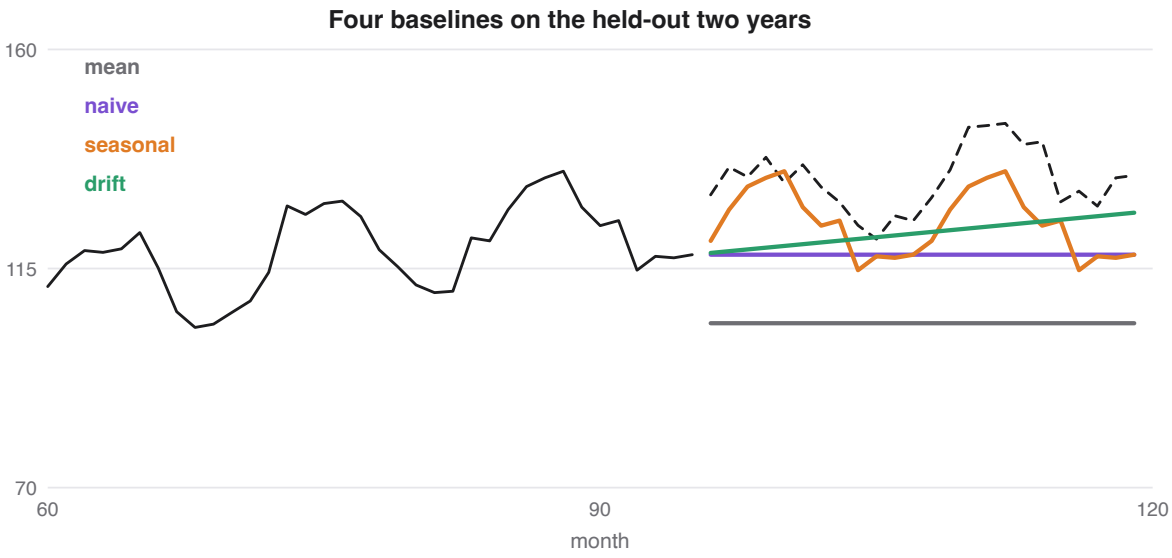
A **trend** is a long-term rise or fall. A **season** is a pattern with a fixed period, such as a month or a weekday. A **cycle** rises and falls with no fixed period. A series with none of these is close to noise.

A3 Four baselines that need no training

To forecast h steps ahead from $y_1, \dots, y_T$:

Method	Forecast
Mean	$\frac{1}{T}\sum_t y_t$
Naive	y_T
Seasonal naive	the value one period earlier
Drift	$y_T + h\,\frac{y_T - y_1}{T-1}$

Insight. A model that cannot beat these has learnt nothing useful.



Each baseline captures something different. The seasonal naive keeps the shape but misses the rise. Drift follows the rise but misses the shape.

A4 Measuring forecasts

With $e_t = y_t - \hat{y}_t$:

$$\text{MAE} = \text{mean}\,|e_t|, \quad \text{RMSE} = \sqrt{\text{mean}\,e_t^2}, \quad \text{MAPE} = 100 \cdot \text{mean}\,|e_t / y_t|.$$

MAPE breaks when y is near zero.

A5 Split in time

> Careful. Never shuffle a series into random folds. The model would train on the future. Use **time-series cross-validation**: train on everything up to an origin, score the forecast h steps later, then move the origin forward and average.

A6 Stationarity and autocorrelation

A **stationary** series looks the same at any point in time, so a series with a trend or season is not stationary. The **autocorrelation** at lag k is

$$r_k = \frac{\sum_t (y_t - \bar{y})(y_{t+k} - \bar{y})}{\sum_t (y_t - \bar{y})^2}.$$

For white noise, about 95% of the values lie within $\pm 1.96/\sqrt{n}$.

A random walk has very high autocorrelation at small lags, but its first difference is close to white noise.

A7 Differencing

The first difference $y_t - y_{t-1}$ turns a linear trend into a constant. The seasonal difference $y_t - y_{t-m}$ removes a season of period m.

A8 Autoregression

Autoregression is linear regression of a value on its own lags:

$$y_t = c + \varphi_1 y_{t-1} + \cdots + \varphi_p y_{t-p} + \varepsilon_t.$$

Each row of X is $[1, y_{t-1}, \ldots, y_{t-p}]$, and the target is y_t, so ordinary least squares gives the coefficients.

A9 Recursive forecasts

A one-step forecast uses the true lags. Forecasting further ahead feeds each forecast back in as a lag, so errors compound, and a stationary model's forecast decays towards its mean $c/(1 - \sum_j \varphi_j)$.

A10 Recurrent networks

A **recurrent network** learns its own summary of the past, a hidden state:

$$h_t = \tanh(W h_{t-1} + U x_t + b), \qquad y_t = V h_t + c.$$

The same weights act at every step, so any sequence length works. **Worked example.** With $W = 0.8$, $U = 1$ and one input pulse, the hidden state goes $0.762,\ 0.544,\ 0.409,\ 0.316,\ 0.248$, a memory that fades.

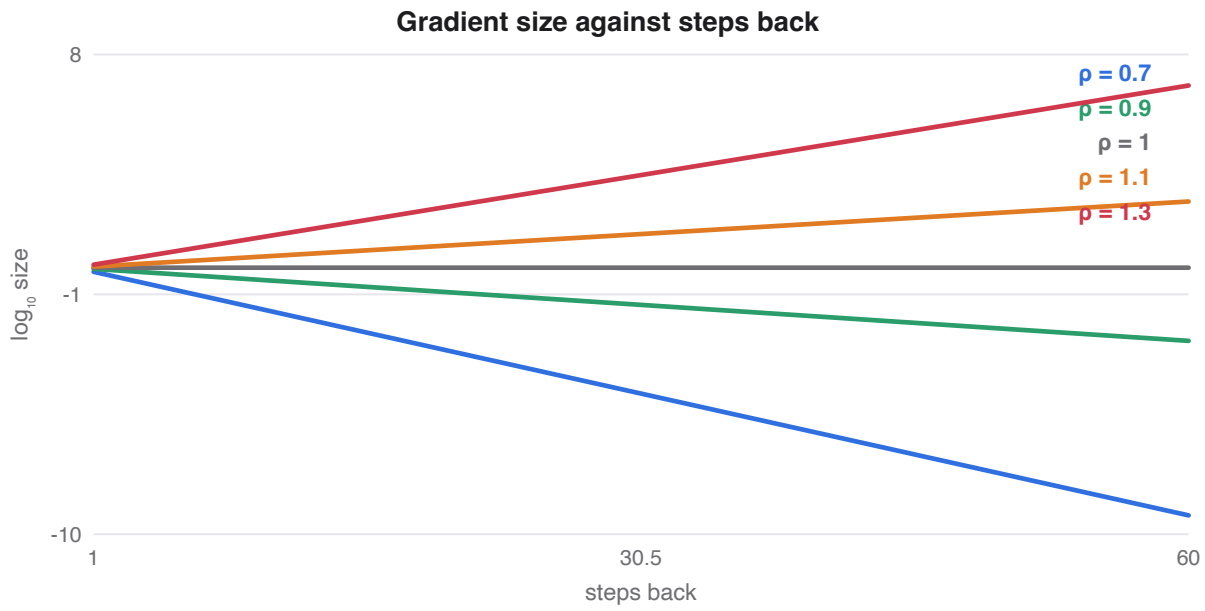
Training unrolls the network through time and applies backpropagation.

A11 Vanishing and exploding gradients

The gradient across k steps is a product of Jacobians:

$$\frac{\partial h_T}{\partial h_{T-k}} = \prod_{j=T-k+1}^{T} \operatorname{diag}(1 - h_j^2)\, W.$$

The growth rate is set by the spectral radius of W, not its norm. Above 1 gradients can explode, and below 1 they vanish, and the $\tanh$ slopes shrink them further.



Small changes in the spectral radius make gradients vanish or explode exponentially with the number of steps.

LSTMs and GRUs use gates to control memory. An LSTM adds to a separate cell state, $c_t = f_t \odot c_{t-1} + i_t \odot g_t$, so along that path the gradient is scaled by the gate rather than by repeated multiplication.

A12 When an autoregression is stationary

For $y_t = \phi y_{t-1} + \varepsilon_t$, expanding gives $y_t = \sum_k \phi^k \varepsilon_{t-k}$, which converges only if $|\phi| < 1$. Then

$$\mathbb{E}[y] = 0, \qquad \operatorname{Var}(y) = \frac{\sigma^2}{1 - \phi^2}, \qquad \rho_k = \phi^k.$$

$\phi = 1$ is a random walk, whose variance grows without limit. Differencing it gives white noise.

A13 ARIMA in one view

An **ARIMA(p,d,q)** model differences a series d times, then combines p autoregressive terms and q past errors. Moving-average terms have $\rho_k = 0$ past lag q, while autoregressions decay gradually, which guides the choice of orders. Seasonal versions apply the same structure at the seasonal lag.

A14 Backpropagation through time, derived

With $h_t = f(h_{t-1}, x_t)$ and loss $L = \sum_t L_t$,

$$\frac{\partial L_T}{\partial h_k} = \frac{\partial L_T}{\partial h_T} \prod_{t=k+1}^{T} \frac{\partial h_t}{\partial h_{t-1}}.$$

A product of $T - k$ Jacobians is why gradients vanish or explode. **Truncated** backpropagation limits this product to the last K steps, for memory.

A15 Walk-forward validation

Train up to time t, forecast the next h steps, then move t forward and repeat. This gives a realistic estimate, including how the model ages, and uses each point as a test once.

Part B In Code

B1 Baselines

```
naive = np.repeat(y[-1], h)
seasonal = y[-m:][np.arange(h) % m]
drift = y[-1] + (np.arange(1, h + 1)) * (y[-1] - y[0]) / (len(y) - 1)
```

B2 Autoregression by least squares

```
X = np.column_stack([np.ones(n - p)] + [y[p - k - 1:n - k - 1] for k in range(p)])
coef = np.linalg.lstsq(X, y[p:], rcond=None)[0]
```

B3 A recurrent cell

```
h = torch.tanh(W @ h + U @ x + b)
y = V @ h + c
```

The notebook **Autoregressive models** in this section fits these on real series.

B4 An AR(1) check

```
import numpy as np

rng = np.random.default_rng(0)
phi, n = 0.8, 200000
y = np.zeros(n)
for t in range(1, n):
    y[t] = phi * y[t - 1] + rng.normal()
print('variance', y.var().round(3), 'theory', round(1 / (1 - phi ** 2), 3))
print('lag 2 correlation', np.corrcoef(y[:-2], y[2:])[0, 1].round(3), 'theory', phi ** 2)
assert abs(y.var() - 1 / (1 - phi ** 2)) < 0.1
```

B5 A complete LSTM forecaster

```
import numpy as np
import torch
from torch import nn

torch.manual_seed(0)
t = np.arange(1200)
series = np.sin(2 * np.pi * t / 24) + 0.1 * np.random.default_rng(0).normal(size=1200)
train = series[:1000]
mu, sd = train.mean(), train.std()
z = torch.tensor((series - mu) / sd, dtype=torch.float32)
W = 48
X = torch.stack([z[i:i + W] for i in range(len(z) - W)]).unsqueeze(-1)
Y = z[W:]
split = 1000 - W


class Forecaster(nn.Module):
    def __init__(self):
        super().__init__()
        self.rnn = nn.LSTM(1, 32, batch_first=True)
        self.out = nn.Linear(32, 1)

    def forward(self, x):
        return self.out(self.rnn(x)[0][:, -1]).squeeze(1)


m = Forecaster()
opt = torch.optim.Adam(m.parameters(), lr=3e-3)
for epoch in range(120):
    i = torch.randint(split, (128,))
    loss = ((m(X[i]) - Y[i]) ** 2).mean()
    opt.zero_grad(); loss.backward()
    nn.utils.clip_grad_norm_(m.parameters(), 1.0)
```

```
    opt.step()

with torch.no_grad():
    mae = (m(X[split:]) - Y[split:]).abs().mean().item() * sd
naive = np.abs(series[1000 + 24:] - series[1000:-24]).mean()
print('LSTM MAE', round(mae, 3), 'seasonal naive', round(naive, 3))
assert mae < 2 * naive
```

B6 Walk-forward evaluation

```
import numpy as np

def walk_forward(series, fit_predict, start, h=1, step=1):
    errors = []
    for t in range(start, len(series) - h + 1, step):
        errors.append(np.abs(fit_predict(series[:t], h) - series[t:t + h]).mean())
    return float(np.mean(errors))

last_value = lambda past, h: np.repeat(past[-1], h)
print(walk_forward(np.arange(20.0), last_value, 10))
assert walk_forward(np.arange(20.0), last_value, 10) == 1.0
```

Part C Practice

P1. It is January, the period is 12, and you want a seasonal naive forecast for March. Which value do you use?

> Last March.

P2. Sales $3, 5, 4, 6, 5, 7, 6, 8$. A model uses the last two days with $\varphi_1 = 0.5$, $\varphi_2 = 0.5$, $c = 0$. What does it predict next?

> $0.5 \times 8 + 0.5 \times 6 = 7$.

P3. Why can you not use random folds for a series?

> They would train on the future and test on the past, giving an over-optimistic score.

P4. Is a random walk stationary?

> No, although its first difference is.

P5. An AR(1) has $c = 2$ and $\varphi = 0.5$. Where does a long recursive forecast settle?

> $2/(1 - 0.5) = 4$.

P6. Why do LSTMs remember longer?

> The cell state is updated by addition with gates, so gradients along it are not repeatedly multiplied by a shrinking factor.

P7. For $\phi = 0.5$ and $\sigma = 1$, find the variance of an AR(1).

$1/0.75 = 1.333$.

P8. What is the lag-3 autocorrelation with $\phi = 0.5$?

0.125.